CS2109S CHEATSHEET

Agents and environments

PEAS Performance measure; Environment; Actuators; Sensors.
Observability Full: complete relevant state. Partial: hidden state.
Dynamics Static: unchanged while deciding. Dynamic: changes. Semi-dynamic: state fixed, score changes with time.
Other axes Single/multi-agent; deterministic/stochastic; episodic/sequential; discrete/continuous; known/unknown rules.
Agent types Reflex: condition–action rule. Goal-based: seek goal. Utility-based: rank outcomes. Learning: improve from experience.
Note: Known rules do not imply fully observable state. In an unknown environment, balance exploration and exploitation.

Search problem

Specify States S; start s_0; goal test $G(s)$; actions $A(s)$; transition $T(s,a)$; step cost $c(s,a,s')$.
Node State + parent/path + cumulative cost $g(n)$. Branching factor b: successors per expansion.
Complete Finds a solution whenever one exists.
Optimal Returns a minimum-cost solution. Equal step costs make minimum depth equivalent.

Systematic search

BFS FIFO; smallest depth. Complete for finite b; optimal for equal costs. Exponential time/space.
IDS Depth-limited DFS at $0, 1, 2, \ldots$. $O(b^d)$ time, $O(bd)$ DFS space. Complete for finite b and solution depth; optimal for equal positive costs.
UCS Priority by $g(n)$. Complete if $b < \infty$, every cost $\geq \epsilon > 0$, and $C^* < \infty$; optimal with nonnegative costs. Same as A, but heuristic is set to 0
Greedy Priority by $h(n)$; ignores cost already paid; generally not optimal.
A Priority by $g(n) + h(n)$; UCS when $h = 0$; usually exponential space.

$$f_{\text{BFS}} = d, \quad f_{\text{UCS}} = g$$

$$f_{\text{greedy}} = h, \quad f_{\text{A}^*} = g + h$$

Note: For cost-based search, accept a goal when removed from the priority queue, not when first generated.

A heuristics

$$0 \leq h(n) \leq h^*(n) \quad \text{(admissible)}$$

$$h(n) \leq c(n,a,n') + h(n') \quad \text{(consistent)}$$

$h^*(n)$ True cheapest remaining path cost. Require $h(\text{goal}) = 0$.
Guarantees Admissible h: optimal A tree search. Consistent h: optimal graph search with a closed set; consistency implies admissibility.
Dominance Among admissible heuristics, if $h_1(n) \geq h_2(n)$, $h_1(n)$ is more informed.
Construction Solve a relaxed problem; precompute a pattern database; learn an estimate. The relaxed optimum is a lower bound.
Note: With an inconsistent heuristic, graph search may need to reopen improved nodes.

Local search

Idea Keep a current state, generate neighbours, and return a state rather than a path. Low memory; usually incomplete/suboptimal.

$$s' = \arg\max_{u \in N(s)} \text{eval}\,(u)$$

Hill climbing If eval $(s') \leq$ eval (s), stop; otherwise set $s \to s'$.
Failure modes Local maximum: better than neighbours, not global. Plateau/shoulder: no immediate improvement.
Escapes Random restarts; simulated annealing may sometimes accept a worse move.

Minimax and alpha–beta

Setting Finite two-player zero-sum perfect-information game; utility from MAX's viewpoint.

$$V(s) = \begin{cases} U(s), & s \text{ terminal} \\ \max_{s'} V(s'), & s \text{ MAX} \\ \min_{s'} V(s'), & s \text{ MIN} \end{cases}$$

Cost Minimax time $O(b^m)$, depth-first space $O(bm)$; optimal against optimal play.
Alpha–beta α: MAX's best lower bound; β: MIN's best upper bound. Prune when $\alpha \geq \beta$. Same root move/value.
Ordering Good moves first increase pruning; ideal time $O(b^{m/2})$.
Cutoff Stop at depth limit; score nonterminal states with a cheap evaluation function. **EVALUATION IS NOT EQUAL TO FINAL SCORE/ TERMINAL NODE**

Learning setup and metrics

Supervised Labelled $(x^{(i)}, y^{(i)})$ pairs, assumed IID. Regression: real target. Classification: discrete class.
Unsupervised Find structure in unlabelled inputs.

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

Interpret Precision: fraction of predicted positives correct. Recall: fraction of actual positives found.
Data split Train fits parameters; validation selects settings; untouched test estimates generalisation.

Entropy and decision trees

$$H(Y) = -\sum_y p(y) \log_2 p(y), \quad 0 \log 0 = 0$$

Note: The log base has to be 2

$$H(Y \mid X) = \sum_x p(x) H(Y \mid X = x)$$

$$IG(Y; X) = H(Y) - H(Y \mid X)$$

Binary entropy For positive fraction q: $H = -q \log_2 q - (1-q) \log_2 (1-q)$. Pure: 0; balanced: 1 bit.
Tree Internal node tests an attribute; branch is an outcome; leaf predicts a label. Split on highest immediate IG.
DTL bases No rows → parent majority; one label → that label; no attributes → current majority. Else split and recurse.
Overfitting A tree can fit finite consistent categorical examples perfectly, including noise. Limit depth or require a minimum leaf size.
Real data Bin continuous values; choose a rule for missing values.

Linear regression

Intercept Set $x_0 = 1$; $X \in \mathbb{R}^{n \times (d+1)}$, $w \in \mathbb{R}^{d+1}$, $Y \in \mathbb{R}^n$.

$$h_w(x) = w^T x = \sum_{j=0}^{d} w_j x_j$$

$$J(w) = \frac{1}{2n} \| Xw - Y \|_2^2$$

$$w = (X^T X)^{-1} X^T Y$$

Normal equation Inverse form needs full column rank; **inversion costs roughly** $O(d^3)$. If singular, remove redundant features
MAE $\frac{1}{n} \sum_i | \hat{y}^{(i)} - y^{(i)} |$.
Note: The $1/2$ in J only simplifies derivatives. Compute every gradient component from the same old w.

Gradient descent and convexity

$$w_i \leftarrow w_i - \eta \frac{\nabla J(w_i)}{\nabla w_i}$$

Note: J here refers to the loss function → taking the partial derivative of it relative to w_i
Rate Large η: overshoot/diverge. Small η: slow.
Batch size Full batch: n examples/update. Mini-batch: b. SGD: 1. Updates/epoch: $\lceil n/b \rceil$.
Tradeoff Smaller batches give cheaper, noisier updates; noisy steps do not guarantee a global minimum.

$$J(tu + (1-t)v) \leq tJ(u) + (1-t)J(v)$$

Convexity For $0 \leq t \leq 1$. If J is twice differentiable on a convex domain, it is convex iff $\nabla^2 J(w) \succcurlyeq 0$ everywhere. Every local minimum is global.
Strict convexity The inequality is strict for $u \neq v$ and $0 < t < 1$; $\nabla^2 J(w) \succ 0$ everywhere is sufficient, but not necessary. A minimiser, if it exists, is unique.
Linear MSE Here $\nabla^2 J = X^T X / n \succcurlyeq 0$; strict convexity holds iff X has full column rank. Penalising every weight with $\lambda \| w \|_2^2$ $(\lambda > 0)$ also makes it strict.

Scaling and feature maps

$$x'_{\text{min max}} = \frac{x - \min x}{\max x - \min x}$$

$$x'_{\text{std}} = \frac{x - \mu}{\sigma}$$

Why scale? Uneven scales cause zig-zag GD updates. Fit scaling on training data; reuse on validation/test. Handle constant features separately.
Feature map $\phi : \mathbb{R}^d \to \mathbb{R}^M$, e.g. $\phi(x) = (1, x, x^2)$.
Effect $w^T \phi(x)$ may be nonlinear in raw x but remains linear in w; a fixed map preserves convexity of linear-MSE/logistic-BCE in w.

Logistic classification

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

Model $p = h_w(x) = \sigma(w^T x)$ estimates class-1 probability. At threshold 0.5, boundary is $w^T x = 0$.

$$\ell_{\text{BCE}}(y,p) = -y\log p - (1-y)\log(1-p)$$

$$J(w) = \frac{1}{n}\sum_i \ell_{\text{BCE}}(y^{(i)}, \sigma(w^T x^{(i)}))$$

Gradient $\nabla J(w) = X^T(p-Y)/n$. Logistic + BCE is convex in w for fixed features; sigmoid alone is not globally convex.

Threshold $p \geq \tau$ predicts class 1; boundary $w^T x = \log(\tau/(1-\tau))$.

Note: For $y=1$, BCE is $-\log p$; for $y=0$, it is $-\log(1-p)$. A confident wrong prediction is costly.

Multiclass vs multilabel

Multiclass Exactly one of K classes. One-vs-one: $\binom{K}{2}$ classifiers + votes. One-vs-rest: K classifiers + highest score.

Multilabel Any subset; $y \in \{0,1\}^K$. Binary relevance: one independent classifier/label; ignores dependencies.

Powerset Treat each observed label set as one class; retains combinations but may create many sparse classes.

Generalisation and tuning

Underfit Too little capacity → high bias.

Overfit Good training fit, poor unseen-data fit; often high variance across training samples.

Parameters Learned weights. **Hyperparameters:** rate, batch size, feature map, tree depth.

Tune Grid, random, or local search on validation data; use held-out test only for final assessment.

Improve Relevant low-noise data, suitable model capacity, pruning/regularisation.